

Department of Computer and Software Engineering**SE: Machine Learning**

Course Instructor:	Date:
Day:	Semester:

Lab 7: Agglomerative Clustering and Dendrograms

Lab Title	CLO	BT	PLO	Weightage
Agglomerative Clustering and Dendrograms				

Name	Roll Number	Report /10	Viva /5	Total /15
Zunaira Abdul Aziz	BSSE23058			

Checked on: _____

Signature: _____

1 Learning Outcomes

After completing this lab, students will be able to:

- Understand hierarchical clustering.
- Implement agglomerative clustering from scratch.
- Compare different linkage methods.
- Interpret dendrograms.

2 Dataset

We will use a synthetic dataset generated using:

`sklearn.datasets.make_blobs`

Dataset properties:

- Number of samples: 100
- Number of clusters: 3
- Features: 2 (for visualization)

3 Agglomerative Clustering

Agglomerative clustering is a bottom-up hierarchical clustering method.

Algorithm:

1. Start with each point as its own cluster.
2. Find the closest pair of clusters.
3. Merge them into a single cluster.
4. Repeat until desired number of clusters is reached.

4 Linkage Methods

Different linkage methods define cluster distance differently.

4.1 Single Linkage

$$d(C_1, C_2) = \min ||x - y||$$

Produces chain-like clusters.

4.2 Complete Linkage

$$d(C_1, C_2) = \max ||x - y||$$

Produces compact clusters.

4.3 Average Linkage

$$d(C_1, C_2) = \text{mean}||x - y||$$

Balanced clustering approach.

5 Part A: Distance Matrix (3 Marks)

Implement: `compute_distance_matrix(X)`

Requirements:

- Compute pairwise Euclidean distances
- Output should be an NxN matrix
- Matrix must be symmetric
- Diagonal values must be zero

def compute_distance_matrix(X):

n = len(X)

dist_matrix = np.zeros((n, n))

for i in range(n):

for j in range(i + 1, n):

Euclidean distance: sqrt(sum((x-y)^2))

dist = np.linalg.norm(X[i] - X[j])

dist_matrix[i, j] = dist

dist_matrix[j, i] = dist # Matrix must be symmetric

return dist_matrix

6 Part B: Linkage Distance (4 Marks)

Implement:

`linkage_distance(cluster1, cluster2, X, method)`

Support the following methods:

- Single linkage
- Complete linkage
- Average linkage

```
def linkage_distance(cluster1, cluster2, X, method="single"):
```

```
    distances = []
```

```
    for idx1 in cluster1:
```

```
        for idx2 in cluster2:
```

```
            dist = np.linalg.norm(X[idx1] - X[idx2])
```

```
            distances.append(dist)
```

```
    if method == "single":
```

```
        return np.min(distances)
```

```
    elif method == "complete":
```

```
        return np.max(distances)
```

7 Part C: Find Closest Clusters (3 Marks)

Implement:

`find_closest_clusters(clusters, X, method)`

Steps:

- Compute distance between all cluster pairs
- Return indices of closest clusters
- Return the corresponding distance

```

def find_closest_clusters(clusters, X, method):
    min_dist = float('inf')
    closest_pair = (None, None)
    for i in range(len(clusters)):
        for j in range(i + 1, len(clusters)):
            dist = linkage_distance(clusters[i], clusters[j], X, method)
            if dist < min_dist:
                min_dist = dist
                closest_pair = (i, j)
    return closest_pair[0], closest_pair[1], min_dist

class MyAgglomerative:
    def __init__(self, n_clusters=3, linkage="single"):
        self.n_clusters = n_clusters
        self.linkage = linkage
        self.labels_ = None
        self.history_ = []

    def fit(self, X):
        n = len(X)
        clusters = [[i] for i in range(n)]
        current_cluster_ids = list(range(n))
        next_cluster_id = n
        while len(clusters) > self.n_clusters:
            i, j, dist = find_closest_clusters(clusters, X, self.linkage)
            self.history_.append([current_cluster_ids[i], current_cluster_ids[j], dist, len(clusters[i])
+ len(clusters[j])])
            new_cluster = clusters[i] + clusters[j]
            clusters.pop(j)
            clusters.pop(i)
            clusters.append(new_cluster)
            current_cluster_ids.pop(j)
            current_cluster_ids.pop(i)

```

```
next_cluster_id += 1
```

```
self.labels_ = np.zeros(n)
```

```
for label, cluster in enumerate(clusters):
```

```
    for point_idx in cluster:
```

```
        self.labels_[point_idx] = label
```

```
def predict(self, X):
```

```
    return self.labels_
```

8 Part D: Agglomerative Clustering (5 Marks)

Implement:

fit(X)

Steps:

1. Initialize each point as its own cluster
2. Iteratively merge closest clusters
3. Stop when desired number of clusters is reached Store:

- Cluster labels
- Merge history (for dendrogram)

```
def fit(self, X):
```

```
    n = len(X)
```

```
    clusters = [[i] for i in range(n)]
```

```
    current_cluster_ids = list(range(n))
```

```
    next_cluster_id = n
```

```
    while len(clusters) > self.n_clusters:
```

```
        i, j, dist = find_closest_clusters(clusters, X, self.linkage)
```

```
        self.history_.append([current_cluster_ids[i], current_cluster_ids[j], dist,  
len(clusters[i]) + len(clusters[j])])
```

```
        new_cluster = clusters[i] + clusters[j]
```

```
        clusters.pop(j)
```

```
        clusters.pop(i)
```

```
clusters.append(new_cluster)

current_cluster_ids.pop(j)

current_cluster_ids.pop(i)

next_cluster_id += 1


self.labels_ = np.zeros(n)

for label, cluster in enumerate(clusters):

    for point_idx in cluster:

        self.labels_[point_idx] = label
```

9 Part E: Dendrogram (Bonus)

Implement:

plot_dendrogram(history) Steps:

- Convert merge history to linkage format
- Plot dendrogram using scipy

```
from scipy.cluster.hierarchy import dendrogram
```

```
def plot_dendrogram(history):
```

```
    linkage_matrix = np.array(history).astype(float)
```

```
    plt.figure(figsize=(10, 5))
```

```
    dendrogram(linkage_matrix)
```

```
    plt.title("Hierarchical Clustering Dendrogram")
```

```
    plt.xlabel("Sample Index")
```

```
    plt.ylabel("Distance")
```

```
    plt.show()
```


10 Part F: Comparison with sklearn (Bonus)

Use:

`sklearn.cluster.AgglomerativeClustering`

Compare:

- Cluster assignments
- Effect of linkage methods

11 Expected Output

Your program should:

- Perform agglomerative clustering
- Display cluster plots
- Show differences between linkage methods
- (Bonus) Display dendrogram

